



AI 赋能企业供应链可视化分析系统 软件设计说明书

院 系： 计算机学院

班级教师： 苏 卓

年 级： 2024

小组成员： 蔡子濠、田锦城

闫若彤、虞诗颢

目 录

1. 引言	4
1.1 编写目的	4
1.2 项目背景	4
1.3 参考资料	4
2. 总体框架设计	5
2.1 系统逻辑架构	5
2.2 技术栈选型	5
2.3 核心构件与拓扑架构	5
2.3.1 整体系统物理部署与拓扑架构	6
2.3.2 FastAPI 后端服务内部逻辑构件架构	6
3. 核心模块详细设计	7
3.1 数据回放与注入模块	7
3.2 AI 智能推理服务与 XAI 归因模块	7
3.2.1 离线数据预处理与特征工程流水线活动图	8
3.2.2 实时流数据注入与双模 AI 推理决策活动图	9
3.3 可视化监控大屏模块	10
4. 数据库与存储设计	11
4.1 关系型数据库(MySQL)	11
4.2 缓存与性能优化 (Redis)	12
5. 接口设计	13
5.1 HTTP RESTful API	13
5.1.1 流式订单注入接口	13
5.1.2 获取实时大屏 KPI 指标	14
5.1.3 获取历史 KPI 趋势数据	14
5.1.4 获取未来 7 天销量预测趋势	15
5.1.5 获取最近发生的历史告警记录	15
5.2 WebSocket API	16
5.2.1 风险告警与实时统计双向通道	16
5.3 WebSocket API	17
5.3.1 数据流回放与 HMAC 签名安全验证时序图	17
5.3.2 实时订单风险推理与大屏 WebSocket 告警广播时序图	18

5.3.3 实时 KPI 统计数据增量更新与大屏同步时序图	18
5.3.4 未来 7 天销量预测查询与缓存/历史兜底时序图	19
6. 部署架构设计	20

1. 引言

1.1 编写目的

本文档基于《产品需求文档：AI 赋能企业供应链可视化分析系统》编写，旨在从技术实现的角度，详细描述系统的软件架构、核心模块划分、数据结构与接口设计。本文档将作为后端开发工程师、前端开发工程师以及算法工程师进行编码和部署的直接依据。

预期的读者：软件工程验收组，开发小组成员。

1.2 项目背景

项目名称：AI 赋能企业供应链可视化分析系统

项目提出者：苏卓（老师）

项目开发者：蔡子濠、田锦城、闫若彤、虞诗颢

面对用户：企业订单风险检测员

1.3 参考资料

在设计与开发本系统时，主要参考了以下文档、规范与数据集：

项目需求文件：

《产品需求文档：AI 赋能企业供应链可视化分析系统》

公开数据集：

Kaggle - DataCo Smart Supply Chain Dataset

框架与库官方文档：

FastAPI 官方异步开发文档：<https://fastapi.tiangolo.com/>

Vue 3 官方组合式 API 文档：<https://vuejs.org/>

ECharts 官方大屏图表配置指南：<https://echarts.apache.org/>

SQLAlchemy 2.0 官方 ORM 使用指南：<https://www.sqlalchemy.org/>

行业技术规范：

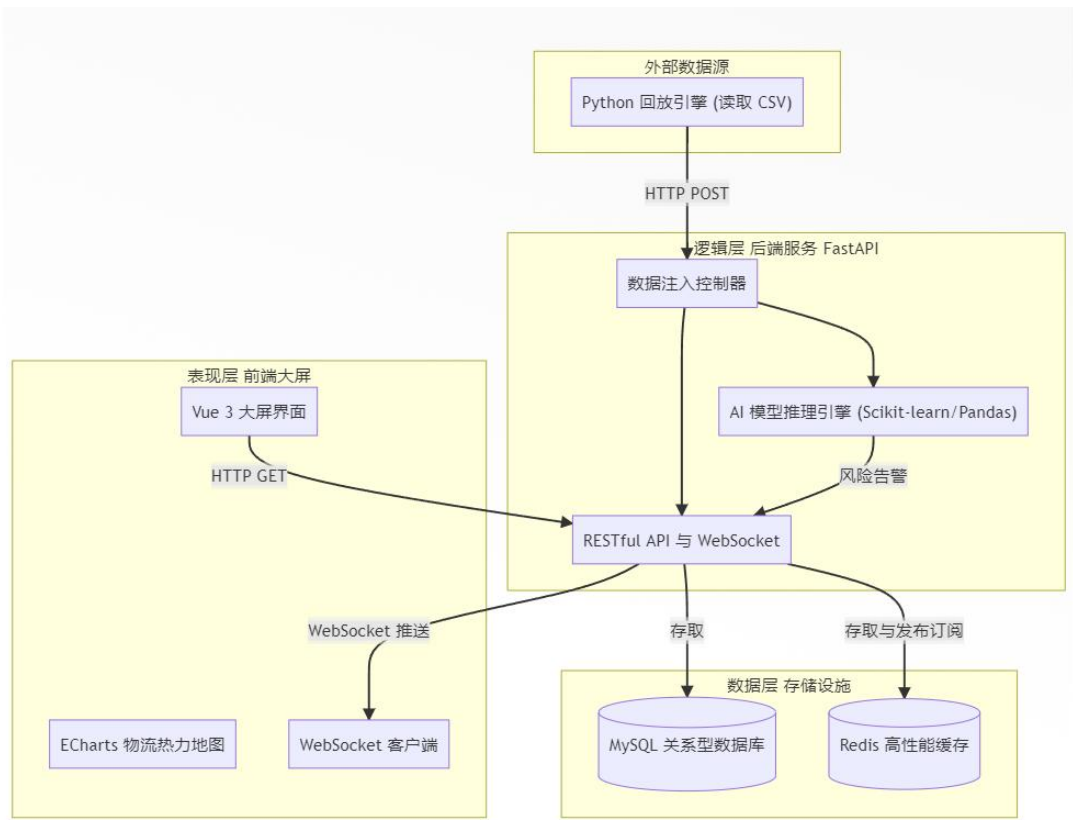
GB/T 8567-2006 《计算机软件文档编制规范》

RFC 2104 《HMAC: Keyed-Hashing for Message Authentication》

2. 总体框架设计

2.1 系统逻辑架构

系统采用前后端分离的微服务雏形架构，分为数据层、逻辑层和表现层。



2.2 技术栈选型

基于 PRD 约束（DC-01 至 DC-05），系统技术栈如下：

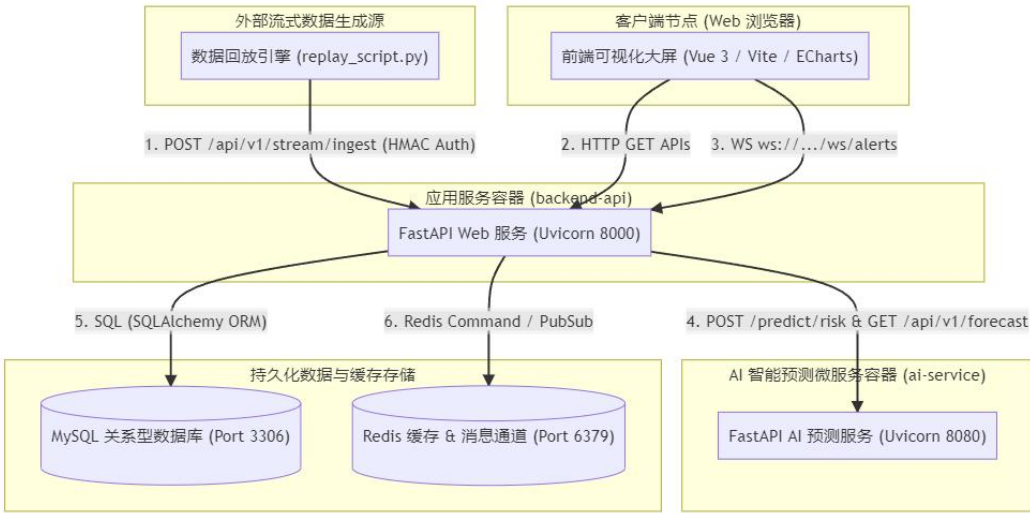
- 前端开发：Vue 3 + Vite + ECharts
- 后端开发：Python 3.10+ + FastAPI
- 算法模型：Scikit-learn, Pandas
- 数据存储：MySQL 8.0, Redis 7.0
- 容器编排：Docker + Docker Compose

2.3 核心构件与拓扑架构

为确保系统开发与物理部署的清晰性，系统架构被细化为物理部署拓扑架构和后端服务内部逻辑架构两部分。

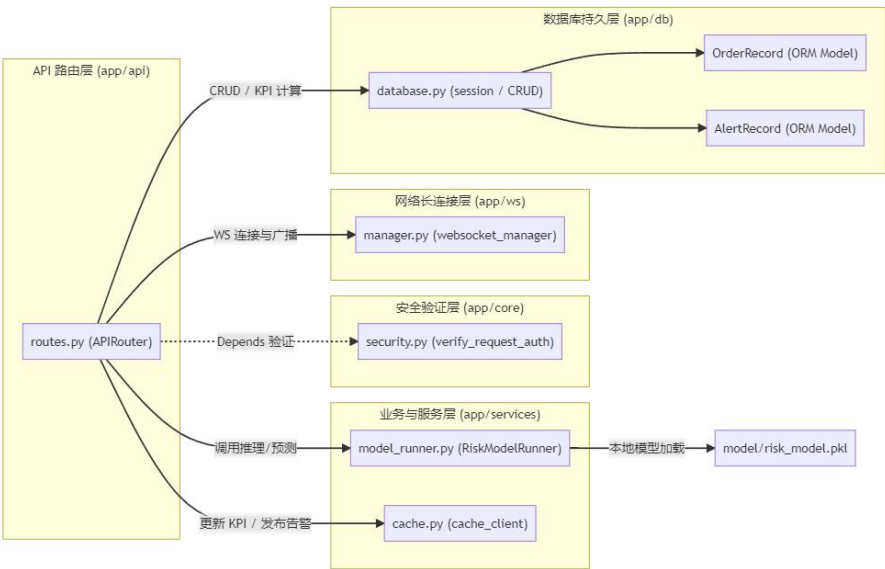
2.3.1 整体系统物理部署与拓扑架构

下面展示了流式数据回放、前端大屏、后端 API 服务、AI 预测服务及底层存储（MySQL、Redis）之间的物理构件连接及所采用的通信协议：



2.3.2 FastAPI 后端服务内部逻辑构件架构

后端核心容器 backend-api 的内部模块设计如下，展示了 API 路由层、安全验证层、业务服务层、Websocket 长连接层以及数据库持久层之间的调用依赖关系：



3. 核心模块详细设计

3.1 数据回放与注入模块

设计思路：独立于主系统后台运行的 Python 脚本 `replay_script.py`，存放在 `data_producer` 文件夹中。

执行流程：

1,按序读取数据：使用 Python 内置的 `csv.DictReader` 流式逐行读取 `DataCoSupplyChainDataset.csv` 数据集，避免一次性加载超大文件造成内存溢出。

2,频率与速率控制：支持通过环境变量 `REPLAY_RATE` 自定义每秒回放订单条数，默认为 5.0 条/秒。根据速率计算等待间隔，通过 `time.sleep()` 实现恒定频率回放，而不用依赖动态时间戳差异计算。

3,数据清洗与构造：针对每行数据仅抽取 `CORE_FIELDS` 中定义的 20 个核心字段（如 `Order Id`, `order date`, `Sales`, `Order Profit Per Order` 等，具体的看 ai 服务需要什么），清洗空白字符后封装为 HTTP JSON 报文体。

4,安全凭证签名与注入：

1，在请求报文头中注入 `X-SCAI-API-Key` 静态密钥。

2，获取当前 UNIX 秒级时间戳作为 `X-SCAI-Timestamp`。

3，计算报文体 SHA256 哈希值，再与 HTTP 方法、请求路径+参数、时间戳串联形成 `canonical string`，使用 `INGEST_HMAC_SECRET` 作为密钥进行 HMAC-SHA256 加密，生成的签名写入报文头 `X-SCAI-Signature`。

4，发送 HTTP 请求到后端。

5,断点续传与异常日志：每次请求成功后，将当前的行号索引和 `order_id` 写入断点文件 `replay_checkpoint.json`，支持下次启动时从断点继续回放。遇到连接超时、网络中断或重试上限超限等失败请求，将失败报文及错误栈追加写入 JSONL 格式的异常日志文件 `failed_orders.jsonl`，确保数据高鲁棒性。

3.2 AI 智能推理服务与 XAI 归因模块

设计思路：

在 FastAPI AI 微服务启动生命周期时，一次性预加载所有 `.pkl` 核心模型文件：

风险拦截分类模型 `risk_model.pkl`

物流拖延预测模型 `delay_model.pkl`

未来 7 天时序销量回归预测模型 `sales_model.pkl`

执行流程：

1,实时推理：

后端 API 接收单条订单数据后，进行字段特征点亮，对齐包含 10 维以上独热特征的输入矩阵。

财务规则拦截：若订单利润为负，直接拦截,根据亏损金额与运输时间的特征

将风险概率设定在 90.0% ~ 98.5%。

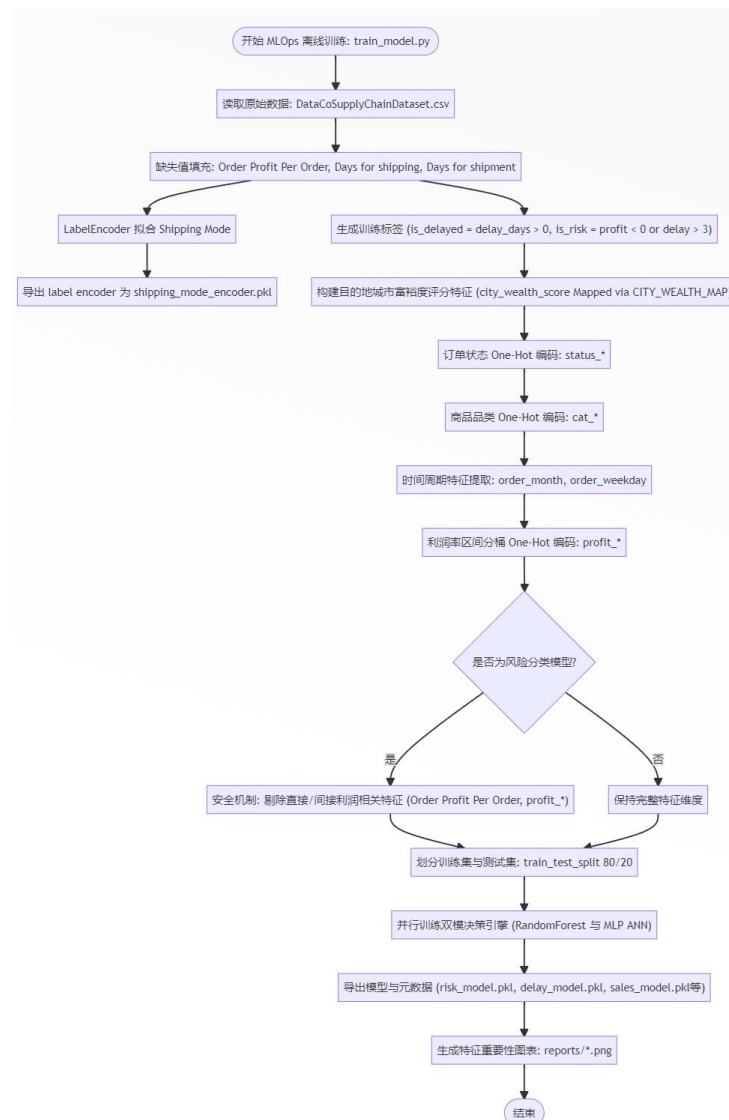
ML 双模综合判定：若非亏损订单，分别调用 risk 风险分类模型延迟预测模型计算概率。若任意概率超过 85%/80%，判定为高风险，通过随机森林的 `feature_importances` 并叠加激活系数（例如金额大则点亮金额权重），生成 Top 5 特征的局部 XAI 解释归因。

销量回归预测：

接收到大屏趋势调用时，从 D+1 到 D+7 天的时序步长内，循环输入时间特征与品类参数，调用 sales 回归模型，提供未来七天销量趋势预测。

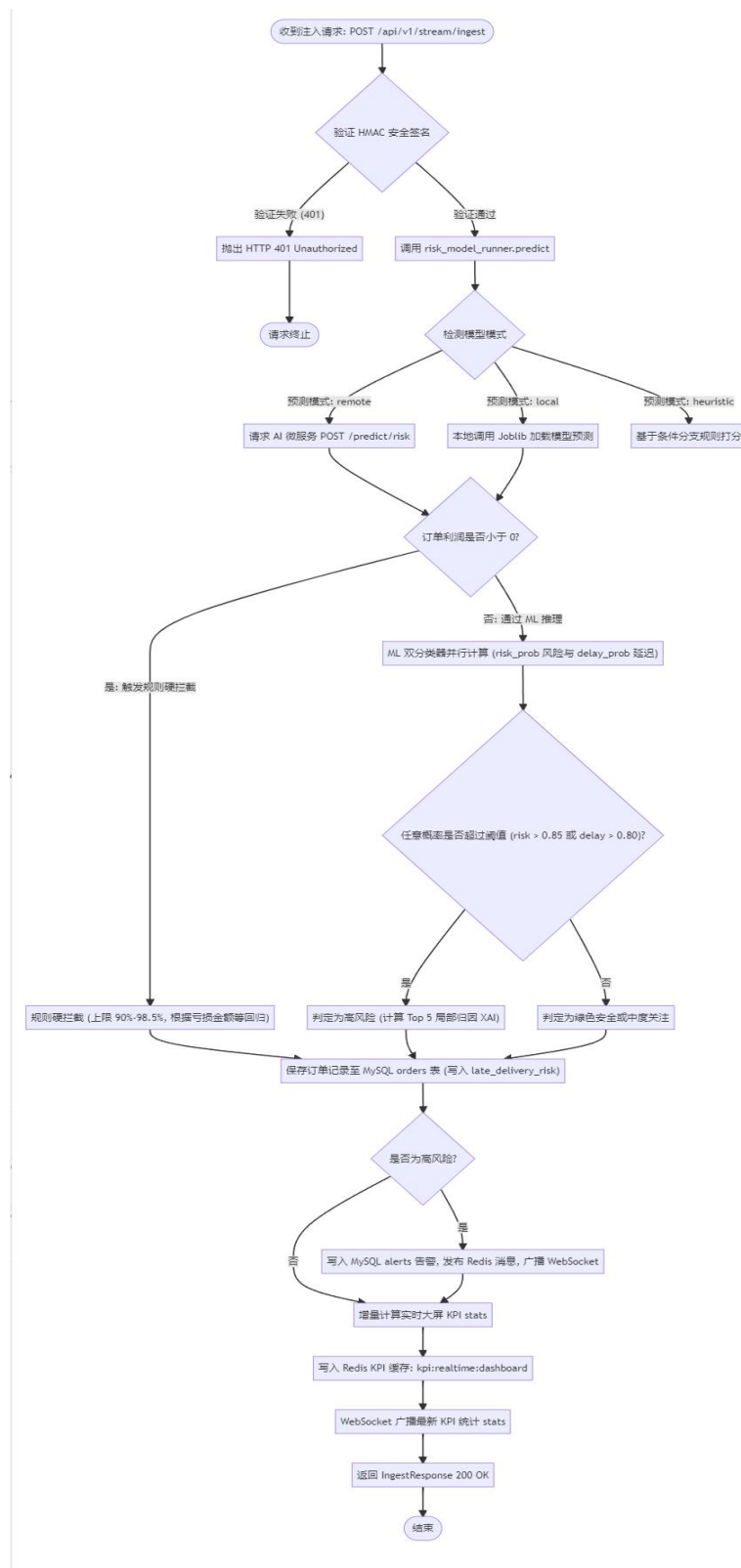
3.2.1 离线数据预处理与特征工程流水线活动图

以下活动图规定了 `train_model.py` 中离线数据加载、清洗、特征独热编码、利润泄露过滤以及最终 RandomForest / MLP ANN 双模型并行训练与导出的完整流水线：



3.2.2 实时流数据注入与双模 AI 推理决策活动图

以下活动图规定了流式订单报文进行 HMAC 签名校验、进入 RiskModelRunner 启动财务规则硬拦截与机器学习双模型(风控/延迟)并行打分, 以及更新 KPI 缓存、发布/订阅广播告警的完整决策链:



3.3 可视化监控大屏模块

设计思路：响应式全屏大屏，包含“数据看板”、“物流热力图”、“历史趋势分析”与“未来销量预测”，并提供右侧的“风险事件追踪”列表。

前端状态与长连接管理：

组件化与状态管理：使用 Vue 3 Setup 语法中的本地响应式变量进行状态管理，并直接集成 ECharts 进行各图表渲染，不采用额外的 Pinia 仓库。

长连接与重连逻辑：建立长连接 WebSocket 到后端接口。若连接断开，采用指数退避算法进行重试。

内存保护与展示限制：接收到告警推送时，将告警插入 warningList 数组头部。为防止大屏持续高频运行引发浏览器内存溢出，界面展示列表长度严格限制为最新的 5 条，旧记录会被丢弃。

4. 数据库与存储设计

4.1 关系型数据库(MySQL)

表 1: orders (供应链订单主表)

字段名	类型	约束	说明
id	INT	PK, Auto_Inc	自增主键 ID
order_id	VARCHAR(64)	Unique, Index	原始订单号
order_date	DATETIME	Index	订单下单时间
shipping_mode	VARCHAR(64)	-	配送运输模式
delivery_status	VARCHAR(64)	-	真实交付状态
customer_city/order_city	VARCHAR(128)	Index	客户城市 / 订单目的地城市
order_status	VARCHAR(64)	-	订单流转状态 (COMPLETE/PROCESSING 等)
latitude / longitude	DECIMAL(10, 6)	-	地理坐标经纬度
order_amount	DECIMAL(12, 2)	-	订单交易总额 (GMV)
order_quantity	INT	-	订购商品数量
scheduled_days	DECIMAL(8, 2)	-	计划配送天数
profit_ratio	DECIMAL(10, 4)	-	订单利润率
risk_score	DECIMAL(5, 4)	-	AI 预测的综合风险得分 (0.0000 - 0.9850)
late_delivery_risk	BOOLEAN	-	AI 是否预测为高拖延风险 (当得分 >= 0.85 时写入 True)
created_at	DATETIME	Index	接收写入时间

表 2: alerts (风险告警明细表)

字段名	类型	约束	说明
id	INT	PK, Auto_Inc	告警事件自增 ID
order_id	VARCHAR(64)	Index	关联的原始订单号
risk_type	VARCHAR(64)	-	主导风险类型 (供应链高欺诈风险/供应

字段名	类型	约束	说明
			链高物流拖延风险)
probability	DECIMAL (5, 4)	-	触发时的确切风险得分
status	INT	-	处理状态 (0: 未确认, 1: 已确认)
detail	TEXT	-	JSON 序列化存储 of XAI 分析结果及 报文
created_at	DATETIME	Index	告警生成时间

4.2 缓存与性能优化 (Redis)

实时 KPI 指标缓存:

Key: kpi:realtime:dashboard (String 格式缓存 JSON)

结构: {"totalOrders": 120, "gmV": 45000.5, "otdRate": 98.2, "riskCount": 8, "delayRate": 95.5}

更新策略: 流数据注入接口每次被调用, 均会增量统计 MySQL 并在后台更新该缓存, 保证大屏刷新性能。

发布订阅队列:

Channel: alerts:channel 广播实时告警消息。

5. 接口设计

5.1 HTTP RESTful API

系统的所有 RESTful API 均使用 /api/v1 作为前缀。

5.1.1 流式订单注入接口

安全验证：必须通过 HMAC-SHA256 签名校验。

入参 (OrderIngestRequest):

```
json
{
  "order_id": "ORD-123456",
  "order_date": "2026-06-20T10:00:00",
  "shipping_mode": "Same Day",
  "delivery_status": "PROCESSING",
  "customer_city": "San Jose",
  "order_city": "San Jose",
  "order_amount": 299.99,
  "order_quantity": 2,
  "scheduled_days": 1.0,
  "profit_ratio": -0.15
}
```

出参 (IngestResponse):

```
json
{
  "status": "success",
  "order_id": "ORD-123456",
  "risk_score": 0.9125,
  "is_high_risk": true,
  "alert": {
    "id": 12,
    "order_id": "ORD-123456",
    "risk_type": "供应链高欺诈/异常风险",
    "probability": 0.9125,
    "status": 0,
    "timestamp": "2026-06-20T10:00:05.123",
    "xai_analysis": {
      "explain_method": "RandomForest MLOps Feature Importance Analysis",

```

```

        "top_features_attribution": {
            "订单利润": 0.752,
            "订单总额": 0.201,
            "运输模式": 0.047
        }
    }
}

```

5.1.2 获取实时大屏 KPI 指标

说明：大屏页面初始化时调用，用于初次渲染指标、订单状态占比、地理热力图等。

出参 (KPIResponse):

```

json
{
    "totalOrders": 125,
    "gmV": 45299.89,
    "otdRate": 98.40,
    "riskCount": 9,
    "delayRate": 95.50,
    "heatMap": [
        { "name": "San Jose", "value": 45 }
    ],
    "regionHeatMap": [],
    "heatMapMeta": {},
    "orderStatus": [
        { "value": 100, "name": "已完成", "itemStyle": { "color": "#91cc75" } },
        { "value": 20, "name": "运输中", "itemStyle": { "color": "#5470c6" } },
        { "value": 5, "name": "待发货", "itemStyle": { "color": "#fac858" } }
    ]
}

```

5.1.3 获取历史 KPI 趋势数据

入参 (Query Params):

hours: 统计历史小时数（默认为 24 小时，限制在 1 到 720 之间）。

出参：返回按小时分组的历史销量及预警点折线趋势。

```

json
{
    "hours": 24,

```

```
    "items": [
      {
        "bucket": "2026-06-21T10:00:00",
        "order_count": 12,
        "risk_count": 2,
        "gmv": 4200.5
      }
    ]
  }
```

5.1.4 获取未来 7 天销量预测趋势

路径：GET /api/v1/forecast

说明：结合传入参数对未来销量进行时序预测。若远程微服务故障，自动调用数据库历史均值作兜底。

入参 (Query Params):

order_amount: 订单基础金额（默认 100.0）

order_quantity: 订单基础件数（默认 1）

shipping_mode: 运输模式（默认 Standard Class）

order_city: 目的地城市（可选）

category: 品类名称（可选）

出参 (ForecastResponse):

```
json
{
  "days": ["D+1", "D+2", "D+3", "D+4", "D+5", "D+6", "D+7"],
  "values": [120.5, 131.0, 128.4, 145.2, 139.1, 142.0, 155.4],
  "daily_forecast_series": [],
  "total_predicted_sales_aggregate": 961.6,
  "avg_daily_sales_volume": 137.37,
  "trend_direction": "上扬成长",
  "source": "ai-service"
}
```

5.1.5 获取最近发生的历史告警记录

入参 (Query Params):

limit: 返回记录的最大限制（默认 50 条，限制在 1 到 200 之间）。

出参:

```
json
{
  "items": [
    {
      "id": 12,
      "order_id": "ORD-123456",
      "risk_type": "供应链高欺诈/异常风险",
      "probability": 0.9125,
      "status": 0,
      "timestamp": "2026-06-20T10:00:05.123",
      "xai_analysis": {}
    }
  ]
}
```

5.2 WebSocket API

5.2.1 风险告警与实时统计双向通道

消息推送报文格式:

异常风险告警 (type: "alert"):

```
json
{
  "type": "alert",
  "data": {
    "id": 12,
    "orderId": "ORD-123456",
    "riskType": "供应链高欺诈/异常风险",
    "probability": 0.9125,
    "level": "danger",
    "icon": "fas fa-exclamation-circle",
    "timestamp": "2026-06-20T10:00:05.123",
    "xai_analysis": {
      "explain_method": "RandomForest MLOps Feature Importance Analysis",
      "top_features_attribution": {
        "订单利润": 0.752,
        "订单总额": 0.201,
```

```
        "运输模式": 0.047
    }
}
}
```

大屏实时统计数据 (type: "stats"):

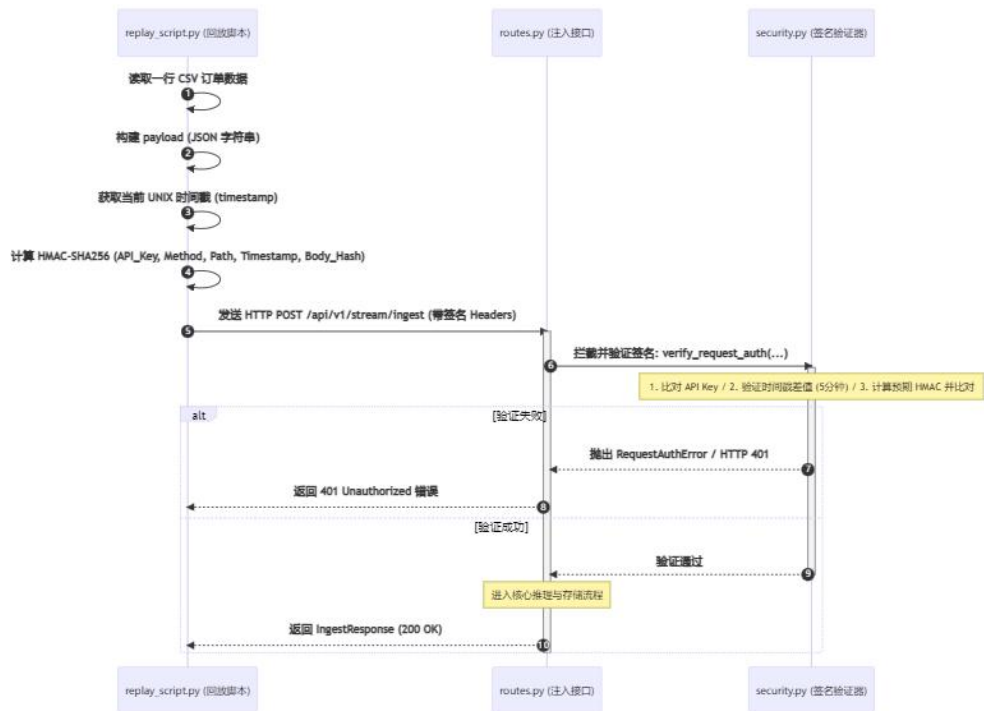
```
json
{
  "type": "stats",
  "data": {
    "totalOrders": 125,
    "gmV": 45299.89,
    "otdRate": 98.40,
    "riskCount": 9,
    "delayRate": 95.50
  }
}
```

5.3 WebSocket API

为确保前后端、算法微服务、缓存与数据库之间通信的一致性，系统核心业务的交互时序图如下：

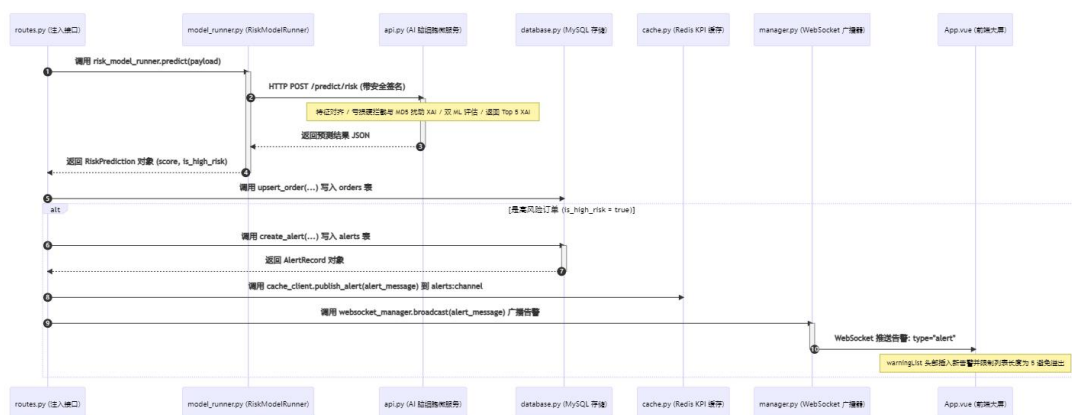
5.3.1 数据流回放与 HMAC 签名安全验证时序图

下面展示了流数据生成器 `replay_script.py` 从本地 CSV 读取订单、构建加密哈希头并通过 FastAPI 后端的安全验证拦截器进行校验的交互时序：



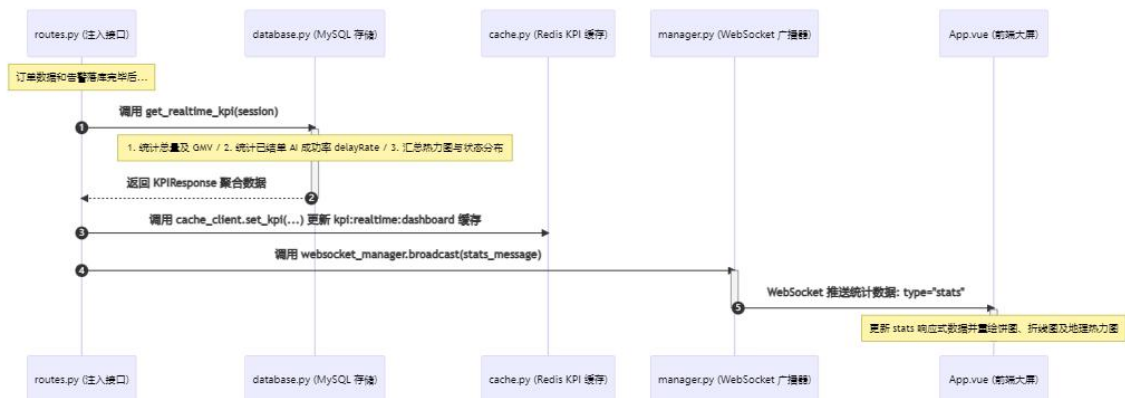
5.3.2 实时订单风险推理与大屏 WebSocket 告警广播时序图

下面展现了在签名通过后，后端如何调用远程 AI 预测微服务进行双模决策、对亏损单硬拦截并添加 Hash 局部扰动，最后落库并触发 WebSocket 实时的告警广播给 Vue 前端：



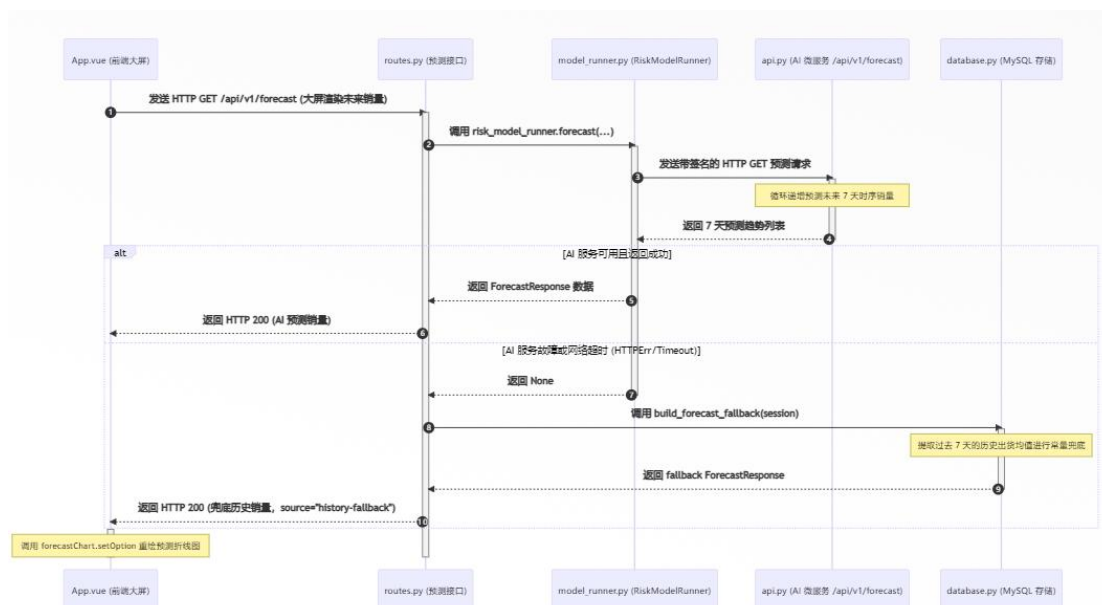
5.3.3 实时 KPI 统计数据增量更新与大屏同步时序图

下面展示了数据每次成功注入后，后端如何对库中已完成订单的预测成功率 delayRate、地理热度等指标进行重新统计，写入 Redis 高速 KPI 缓存，并广播至大屏客户端触发 ECharts 实时重绘：



5.3.4 未来 7 天销量预测查询与缓存/历史兜底时序图

以下展现了 Vue 前端大屏启动或刷新时，调用销量折线预测接口，如何在 AI 服务微服务健康时返回基于 `sales_model.pkl` 的时序销量预测，以及在其异常故障时，无缝切换到 MySQL 历史统计兜底的弹性容灾流程：



6. 部署架构设计

为满足一键演示需求，项目根目录需提供 `docker-compose.yml`，编排以下服务：

mysql-db: MySQL 容器，映射端口 3306，挂载初始化 SQL 脚本用于建库建表。

redis-cache: Redis 容器，映射端口 6379。

backend-api: 基于 `python:3.10-slim` 构建的 FastAPI 容器，启动命令为 `uvicorn main:app --host 0.0.0.0 --port 8000`。映射端口 8000。依赖 `mysql` 与 `redis` 启动。

frontend-dashboard: 基于 Nginx 构建的 Vue 3 静态资源容器。打包后代理转发 `/api` 到 `backend-api:8000`。

data-replay (可选/手动): 数据回放脚本容器，可在其它容器就绪后执行一次性任务或后台常驻执行。